

Machine Learning

Alessio Virgillito

March 2026

Indice

1	Preambolo	2
2	Introduzione al Machine Learning	2
2.1	ML vs Programmazione Tradizionale	2
2.2	La formula magica	3
2.3	Il ciclo di apprendimento della Macchina	3
2.4	Le famiglie di apprendimento nel ML	4
3	I Compiti dell'Apprendimento Supervisionato	4
3.1	La Classificazione - Prevedere una Categoria	4
3.2	La Regressione - Prevedere un Numero	5
3.3	La Regressione Lineare	5
4	L'addestramento e l'Ottimizzazione del Modello	6
4.1	Loss Function	6
4.2	La Discesa del Gradiente (Gradient Descent)	7
4.3	Overfitting e Underfitting	7
4.4	Condizioni di generalizzazione	8
4.5	Suddivisione del Dataset	8
5	Valutazione del Modello e Metriche di Successo	9
5.1	Valutare i modelli di Classificazione	9
5.2	Valutare i modelli di Regressione	10

1 Preambolo

Il seguente documento è stato prodotto con lo scopo di introdurre i concetti basilari della disciplina Machine Learning, Artificial Intelligence e vari aspetti legati a questa materia. Lo scopo dei seguenti appunti è puramente didattico e mirato alla costruzione di una Tesi Sperimentale di Primo Livello (Laurea Triennale in Informatica). L'argomento della Tesi mira all'impiego di soluzioni tramite ML, DL (Deep Learning), LLM (Large Language Model) e AI al mondo giuridico, della sentenza e delle scelte di un giudice artificiale su un caso reale. Per questo motivo gli esempi introdotti sono tutti legati a questo argomento.

2 Introduzione al Machine Learning

Il Machine Learning, o apprendimento automatico, è una branca dell'Intelligenza Artificiale che permette ai computer di imparare dai dati invece di seguire istruzioni rigide pre-programmate. Come vedremo, invece che scrivere milioni di righe di codice per ogni possibile scenario, si addestra un **modello** che individua autonomamente *schemi e regole* nei dati per fare previsioni o prendere decisioni.

2.1 ML vs Programmazione Tradizionale

Se dovessimo pensare alla costruzione di un programma che risolva un problema, nella Programmazione Tradizionale, questo verrebbe affrontato in un modo semplice.

- Dati e regole in input → Risposte in Output.

Dunque il programmatore scrive codice per ottenere ciò che desidera in output. Tuttavia, cosa accadrebbe se ciò che vogliamo analizzare ha molteplici vie? Se ad esempio volessimo scrivere un programma che analizzi delle casistiche legali: *"SE il danno è sotto i 500 euro E il contratto non è stato firmato, ALLORA il risarcimento è negato."* Dovremmo scrivere un codice infinito che, oltre a non rispettare una buona norma di programmazione, rischia quasi sempre di lasciare alcuni buchi (nel caso dell'esempio, sarebbe impossibile prevedere tutte le casistiche legali). Notiamo subito che il rischio in applicazioni reali è molto grande.

Con il Machine Learning, che in molte occasioni abbrevieremo come ML, il processo visto con la programmazione tradizionale è completamente opposto. Infatti, nel ML, diamo in pasto al sistema migliaia di **Dati** (ad esempio migliaia di cause passate) e le **Risposte** (nell'esempio di prima le sentenze dei giudici veri). Il sistema allora si "studia" i casi e capisce da solo quali sono le regole e i pattern che portano a una certa sentenza. Dunque:

- Dati e Risposte in input → Regole (il modello) in Output

2.2 La formula magica

Alla base di tutto il Machine Learning, c'è il tentativo di trovare una funzione matematica, che chiamiamo f , in grado di trasformare un input in un output.

- $x = \text{Input}$ (chiamati anche *Features* o *caratteristiche*). Nell'esempio giuridico potrebbero essere i testi delle denunce, le prove, gli articoli di legge citati.
- $y = \text{Output}$ (chiamata anche *Label* o *etichetta*). Nell'esempio giuridico, è la decisione del giudice. Es. "Colpevole/Innocente" oppure "Risarcimento accordato/negato".

L'obiettivo del ML è **addestrare** un modello per trovare la migliore funzione f possibile, in modo che quando gli verrà data un nuovo dato x in input (una nuova causa ad esempio), *mai vista prima*, lui saprà prevedere un output y estremamente accurato.

2.3 Il ciclo di apprendimento della Macchina

Abbiamo compreso che la Macchina adesso diventa intelligente, cercando di non dare Risposte, ma usandole per fornire il modello matematico iniziale. Se dunque alla macchina dobbiamo dare in pasto dei dati in Input, essa come fa a sfruttarli per imparare? Come trova questa famosa funzione f ? Non è di certo magica e ricordiamo che non è cosciente (avremmo allora risolto tutto il lavoro)!. La macchina **procede a tentativi ed errori**, in un ciclo continuo.

In modo estremamente semplificato potremmo suddividere il ciclo di apprendimento della macchina in 4 fasi:

1. Previsione: La macchina prende un input x , tira a indovinare e produce una sua risposta.
2. Calcolo dell'errore (Loss Function): Il sistema confronta la sua risposta con la risposta reale (ad esempio la sentenza reale del giudice) e calcola *quanto ha sbagliato*. Questa misurazione si chiama *Funzione di costo* o *Loss Function*.
3. Aggiornamento (Ottimizzazione): Usando la matematica (nello specifico, un algoritmo chiamato *Gradient Descent*), il modello aggiusta i suoi ingranaggi interni ("i pesi") per far sì che, al tentativo successivo, l'errore sia un po' più piccolo.
4. Il ciclo si ripete migliaia o milioni di volte finché l'errore non diventa piccolissimo. A quel punto, diciamo che il modello è **addestrato**.

Come facciamo imparare la macchina allora?

2.4 Le famiglie di apprendimento nel ML

Esistono tre modi principali per far imparare la macchina:

- Apprendimento Supervisionato (**Supervised Learning**): è quello che abbiamo già descritto. Alla macchina diamo dati con le risposte già pronte (etichettate). È il metodo che, nel caso di un confronto con dataset già disponibili, risulta più utile. (es. Il caso giudiziario).
- Apprendimento NON Supervisionato (**Unsupervised Learning**): Diamo alla macchina tanti dati senza risposte. Le chiediamo: "Trovami dei gruppi o dei pattern logici". Nell'esempio giuridico, raggruppare cause civili simili tra loro senza sapere a priori di cosa parlano.
- Apprendimento per Rinforzo (**Reinforcement Learning**): La macchina impara interagendo con un ambiente, ricevendo un *premio* se fa bene e una *punizione* se sbaglia (è usato molto per i videogiochi o per addestrare i robot a camminare).

3 I Compiti dell'Apprendimento Supervisionato

Come già annunciato nel preambolo, questo tipo di apprendimento è una tecnica di ML in cui i modelli vengono addestrati utilizzando dataset etichettati, composti da coppie di input e output desiderato. L'algoritmo impara a mappare gli input alle risposte corrette, permettendogli di prevedere risultati su nuovi dati non visti. Vedremo che i compiti principali sono **classificazione** (categorie) e **regressione** (valori continui), a seconda di come è fatta la Risposta y che abbiamo citato nel cap. 1.2

Dunque tutto ruota intorno al dataset, con coppie di dati: Input x *feature* e un output y *label*. Rimanendo sempre sull'esempio del caso giuridico, la *feature* o x è **tutto l'insieme dei documenti della causa**, mentre la *label* o y è **la decisione finale** presa storicamente dal giudice.

3.1 La Classificazione - Prevedere una Categoria

Usiamo la Classificazione quando la risposta che cerchiamo (y) è una categoria precisa, un'etichetta. Non ci sono vie di mezzo. L'algoritmo traccia dei "confini" matematici per separare i casi. La classificazione si divide a sua volta in:

- **Classificazione Binaria**: Ci sono solo due opzioni possibili, solitamente indicate con 0 e 1. Banalmente nel nostro esempio ormai avviato, l'IA legge il ricorso e deve dire 0 (Respinto) oppure 1 (Accolto). Un algoritmo classico per fare questo si chiama *Regressione Logistica* (N.B. Non ha a che fare con La Regressione che vedremo dopo, serve per classificare!). Questo algoritmo calcola la probabilità da 0% a 100% che un ricorso, nel nostro es., venga accolto.

- **Classificazione Multiclasse:** Ci sono più di due opzioni. Ad esempio, potremmo voler addestrare l'IA a leggere una sentenza e categorizzarla in: 1 (Diritto di Famiglia), 2 (Diritto del Lavoro), 3 (Diritto Societario).

3.2 La Regressione - Prevedere un Numero

La regressione, tecnica di apprendimento supervisionata come abbiamo visto, modella la relazione tra *feature di input* e una variabile *target continua*, utilizzando metodi statistici per prevedere la variabile target sulla base di nuovi dati di input. I modelli di regressione esaminano un gran numero di variabili, identificando quelle che hanno il maggiore impatto sui risultati. La regressione è fondamentale per il Machine Learning, soprattutto per i casi d'uso **predittivi**. Adattando un modello di regressione ai dati, possono essere sostituite congetture e intuizioni sui fattori che guideranno i risultati e i comportamenti futuri. L'algoritmo non divide i dati in recinti, ma cerca di tracciare una linea che segua l'andamento dei dati per fare una stima quantitativa. Infatti, usiamo la Regressione quando la risposta che cerchiamo y non è una categoria, ma un **numero continuo**, un valore reale.

Vediamo un esempio: Immaginiamo che un' IA *giudicante* abbia deciso che l'imputato è colpevole (usando la classificazione). Ora deve decidere quanto deve pagare. L'ammontare esatto del risarcimento in euro è un numero continuo, potrebbe essere un numero in un range (ad esempio 100€, 540€ o 10.000€). Per fare questa stima si usano algoritmi come la *Regressione Lineare*.

3.3 La Regressione Lineare

In **Statistica**, la Regressione Lineare ha il compito di trovare quella retta che cerca di spiegare, nel miglior modo possibile, tutti i dati. Sappiamo che, *in Statistica*, la formula della regressione é:

$$y = mx + q$$

- y è la variabile indipendente;
- x la variabile dipendente;
- m è la pendenza (coefficiente angolare);
- q è l'intercetta.

E' un modello che ci permette di fare previsioni.

In **Machine Learning**, la formula di regressione lineare si scrive così:

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + q$$

- y (La previsione): E' il valore continuo che vogliamo indovinare. Nel contesto giuridico ad esempio, potrebbe essere la stima esatta in euro di un risarcimento danni.

- $x_1, x_2, \dots, x_n$ (Le feature): Sono le variabili indipendenti (gli input). In una causa civile potrebbero essere: i giorni di prognosi medica, il reddito della vittima, la percentuale di colpa, ecc.
- $w_1, w_2, \dots, w_m$ (I pesi - weights): In statistica erano i coeff. angolari. Nel ML, i pesi indicano **"l'importanza"** che il modello dà a ciascuna feature. Se i giorni di prognosi sono fondamentali per calcolare il danno, il loro peso w sarà molto alto.
- q (il Bias): In statistica era l'intercetta. E' il valore di base della previsione quando tutti gli input x sono zero.

Dunque la differenza tra Statistica e ML nel campo della Regressione Lineare sta nel fatto che, in Statistica, per trovare la retta migliore, si usa una formula "chiusa" ed esatta (es. *Metodo dei Minimi Quadrati*) che calcola i coefficienti in un colpo solo.

Nel ML, quando si ha a che fare con dataset giganteschi, calcolare la formula esatta diventa computazionalmente impossibile per il computer. Quindi, l'IA parte con pesi w e bias q (o b) **scelti a caso**, guarda l'errore che commette, e li "aggiusta" passo dopo passo in **modo iterativo**.

4 L'addestramento e l'Ottimizzazione del Modello

L'addestramento è il processo iterativo attraverso cui il modello impara a mappare gli input x negli output corretti y . Affinché questo avvenga, il sistema ha bisogno di due componenti fondamentali: una metrica per capire quanto sta sbagliando e un meccanismo per correggere i propri errori.

4.1 Loss Function

La funzione di Costo (o Perdita) calcola la **distanza matematica** tra la previsione del modello (che chiameremo $\hat{y}$) e la risposta (ad es. quella del giudice, che chiameremo y). Più questo valore è alto, peggio sta lavorando il modello! A seconda del compito (visto nel Cap.2), si usano funzioni differenti:

- Per la **Regressione**: Si utilizza comunemente l'Errore Quadratico Medio (*MSE - Mean Squared Error*). Penalizza enormemente gli errori grandi elevandoli al quadrato:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

dove n è il numero di casi considerati.

- Per la **Classificazione Binaria**: Si usa la *Log Loss* (o *Binary Cross-Entropy*), che lavora sulle probabilità. Penalizza duramente il modello se è "molto sicuro" di una decisione che in realtà è sbagliata.

4.2 La Discesa del Gradiente (Gradient Descent)

Una volta calcolato l'errore, come facciamo a modificarlo? Entra in gioco l'algoritmo di ottimizzazione più importante del Machine Learning: **La Discesa del Gradiente**. La discesa del gradiente è una tecnica matematica che trova in modo *iterativo* i pesi e il bias che producono il modello con la perdita più bassa. La discesa del gradiente trova il peso e il bias migliori ripetendo la seguente procedura per un numero di iterazioni definite dall'utente.

Il modello inizia l'addestramento con pesi e bias casuali vicini allo zero e poi ripete i seguenti passaggi:

1. Calcola la perdita con il peso e il bias attuali.
2. Determina la direzione in cui spostare i pesi e il bias che riducono la perdita.
3. Sposta i valori di peso e bias di una piccola quantità nella direzione che riduce la perdita.
4. Torna al passaggio 1 e ripeti la procedura finché il modello non riesce a ridurre ulteriormente la perdita.

In altre parole, immaginiamo la Funzione di Costo come una catena montuosa. L'obiettivo dell'algoritmo è scendere a valle, fino a trovare il punto più basso in assoluto (*il minimo globale*), dove l'errore è vicino a zero. Dunque si calcola il gradiente (la derivata prima) della funzione di costo rispetto a ciascun peso w . Il gradiente indica, come detto, la direzione della pendenza massima in salita; l'algoritmo, perciò, aggiorna i pesi muovendosi nella direzione opposta. La formula di aggiornamento di un singolo peso é:

$$w_{nuovo} = w_{vecchio} - \eta \cdot \nabla L$$

dove:

- ∇L è il gradiente calcolato.
- η è il **Learning Rate** (Tasso di Apprendimento). Rappresenta la grandezza del "passo" che il modello fa verso il basso. Se η è troppo grande, il modello rischia di saltare da una parte all'altra della valle senza mai scendere; se è troppo piccolo, ci metterà un'infinità di tempo ad addestrarsi.

4.3 Overfitting e Underfitting

Il vero scopo del Machine Learning non è avere un errore nullo sui dati di addestramento, ma saper generalizzare su casi mai visti. Qui risiedono i due problemi principali:

- **Underfitting:** Il modello è troppo rigido e non riesce a catturare la logica sottostante ai dati (come una retta che cerca di approssimare una curva complessa).
- **Overfitting:** Il modello si adatta in modo così maniacale ai dati di addestramento da impararne a memoria anche il rumore e i dettagli irrilevanti. Di conseguenza, nell'esempio giuridico, fallisce quando deve giudicare una sentenza nuova.

Un modello dunque deve fare buone previsioni su dati *nuovi*. In altre parole, si vuole creare un modello che *si adatti* ai nuovi dati. Un modello **overfit** fa previsioni eccellenti sul set di addestramento, ma previsioni scarse sui nuovi dati. Un **modello sottoadattato** non fa nemmeno buone previsioni sui dati di addestramento. Per fare un esempio reale, se un modello overfit è come un prodotto che funziona bene in laboratorio, ma male nel mondo reale, un modello underfit è come un prodotto che *non funziona bene nemmeno in laboratorio*. Come detto all'inizio del sotto-capitolo, l'obiettivo è creare un modello che si generalizzi bene ai nuovi dati. Dunque la **generalizzazione** è l'opposto dell'overfitting.

4.4 Condizioni di generalizzazione

Un modello viene addestrato su un set di addestramento (che vedremo dopo), ma il vero test del suo valore è la sua capacità di fare previsioni su nuovi esempi, in particolare su dati reali. Durante lo sviluppo di un modello, il set di test funge da sostituto dei dati reali. L'addestramento di un modello che generalizza bene implica le seguenti condizioni del set di dati:

- Gli esempi devono essere **distribuiti in modo indipendente e identico**, che è un modo elegante per dire che i tuoi esempi non possono influenzarsi a vicenda.
- Il set di dati è **stazionario**, il che significa che non cambia in modo significativo nel tempo.
- Le partizioni del set di dati hanno la stessa distribuzione. In altre parole, gli esempi nel set di addestramento sono statisticamente simili agli esempi nel set di convalida, nel set di test e nei dati reali.

4.5 Suddivisione del Dataset

Per combattere l'overfitting e simulare la realtà, il dataset originale di sentenze **non viene mai fornito al modello tutto insieme**, ma viene suddiviso rigorosamente in tre porzioni:

1. **Training Set** (circa 70-80%): I dati su cui il modello si allena calcolando i gradienti e aggiornando i pesi.

2. **Validation Set** (circa 10-15%): Un set di dati usato durante l'addestramento per valutare temporaneamente il modello e calibrare iperparametri come il *Learning Rate* η .
3. **Test Set** (circa 10-15%): I dati finali, completamente invisibili al modello fino alla fine del progetto. Su questi dati si calcolano le metriche di successo da riportare nella tesi.

5 Valutazione del Modello e Metriche di Successo

Una volta che il modello ha terminato la fase di addestramento, è necessario misurarne le reali capacità di generalizzazione utilizzando il **Test Set** (dati mai visti prima del modello). Le metriche di valutazione cambiano radicalmente a seconda che si stia risolvendo un problema di Classificazione o di Regressione.

5.1 Valutare i modelli di Classificazione

Quando l'obiettivo è prevedere una categoria (Classificazione Binaria o Multi-classe), lo strumento fondamentale è la **Matrice di Confusione** (*Confusion Matrix*). E' una tabella che incrocia le previsioni del modello con le etichette reali (la *Ground Truth*).

Nel caso base (due classi: Positiva e Negativa), la matrice evidenzia quattro casistiche:

Nota Bene¹: *Classe Positiva* = La caratteristica che il modello sta **attivamente cercando di rilevare**. L'oggetto principale della nostra ricerca. [spesso indicata con 1]

Classe Negativa = E' l'assenza di quell'evento, ovvero lo stato di "normalità", di default o il resto dei casi. [spesso indicata con 0]

- **Veri Positivi (TP) e Veri Negativi (TN):** Le previsioni corrette.
- **Falsi Positivi (FP - Errore di Tipo 1):** Il modello prevede la classe Positiva, ma il valore reale è negativo.
- **Falsi Negativi (FN - Errore di Tipo 2):** Il modello prevede la classe Negativa, ma il valore reale è Positivo.

Da questi quattro valori derivano le metriche standard:

¹Esempio Classico: **Filtro Anti-Spam**: Se addestri un'IA a bloccare lo spam, l'obiettivo dell'IA è "trovare lo spam". Quindi la Classe Positiva (1) è "L'email è Spam" (anche se lo spam è una cosa fastidiosa!), mentre la Classe Negativa (0) è "L'email è normale".

1. **Accuratezza (Accuracy)**: La percentuale di previsioni corrette sul totale. E' utile solo se le classi nel dataset sono bilanciate (es. 50% positivi, 50% negativi).

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

2. **Precisione (Precision)**: La proporzione di previsioni positive che erano effettivamente corrette. Risponde alla domanda: "*Di tutti i casi etichettati come positivi dal modello, quanti lo sono davvero?*".

$$Precision = \frac{TP}{TP+FP}$$

3. **Richiamo (Recall)**: La proporzione di casi positivi reali che il modello è riuscito a identificare correttamente.

$$Recall = \frac{TP}{TP+FN}$$

5.2 Valutare i modelli di Regressione

Se invece il modello deve prevedere un valore continuo (Regressione), non possiamo usare "*giusto o sbagliato*", perché indovinare un numero con infiniti decimali è impossibile. Dobbiamo misurare quanto la previsione si discosta dal valore reale.

Le metriche principali in questo ambito sono:

1. **MAE (Mean Absolute Error - Errore Assoluto Medio)**: E' la media delle differenze tra le previsioni e i valori veri. E' facile da interpretare (se, come detto prima, vogliamo prevedere prezzi in euro (es. un risarcimento), il MAE fornisce l'errore medio in euro).

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

2. **RMSE (Root Mean Squared Error - Radice dell'Errore quadratico medio)**: Estrae la radice quadrata del MSE (che è stato visto nel Cap. 3). Poiché eleva gli errori al quadrato prima di farne la media, penalizza in modo molto severo gli errori grandi.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

3. **Coefficiente di Determinazione (R^2)**²: E' una metrica statistica che, in %, ci dice quanto i dati spiegano il modello. Indica la proporzione della varianza nella variabile dipendente (y) che è predicibile dalle variabili indipendenti (x). Un $R^2 = 1$ indica previsioni perfette, riusciamo a spiegare il 100% (modello perfetto).

²In Statistica $R^2 = \rho^2$ si calcola trovando prima $\rho = \frac{Cov(X,Y)}{Var(x)Var(y)}$, dove ρ è il *coefficiente di correlazione tra due variabili aleatorie*.